EXERCISE 04: PROGRAMS USING PANDAS — COMPREHENSIVE STUDY GUIDE

> **College Data Science Laboratory Course — Practical Examination Study Guide**
> **Source Code File:** `e04_pandas.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Domain:** Tabular Data Structures, Data Wrangling & Preprocessing

1. Program Information and Aim

- **Program Identity:** Program 04 — `e04_pandas.py` (Contains 12 Core Data Wrangling Exercises).
- **Academic Aim:** To master tabular data manipulation using Python's `pandas` library. The exercises cover: (1) List-to-Series conversion, (2) Date range frequency generation, (3) Dictionary-to-DataFrame creation, (4) 2D List-to-DataFrame creation with column names, (5) CSV file ingestion via `medal.csv`, (6) Multi-column composite sorting with mixed directions, (7) Custom index resetting via `reset_index(drop=True)`, (8) Integer-location positional slicing using `.iloc`, (9) Split-Apply-Combine aggregation using `groupby()` and `mean()`, (10) Missing value imputation using `fillna(0)` on transposed dictionary DataFrames, (11) In-place conditional boolean masking on column data, and (12) Relational database-style inner merging on common keys.
- **Required Libraries & Supporting Files:**
  - `pandas as pd`: Tabular data analysis and manipulation.
  - `numpy as np`: For `np.nan` representation in missing value exercises.
  - `medal.csv`: Local CSV file containing Olympic gold medal records.

2. Prerequisites and Foundational Concepts

- **Core Data Structures:**
  - **`pd.Series`:** A one-dimensional labeled homogeneous array capable of holding any data type, accompanied by an explicit index.
  - **`pd.DataFrame`:** A two-dimensional size-mutable, tabular data structure with labeled axes (rows and columns). Each column is internally a `Series`.
- **Positional Indexing (`.iloc`) vs Label Indexing (`.loc`):** `.iloc[row_idx, col_idx]` accesses rows and columns strictly by 0-based integer position (analogous to Python list slicing). In contrast, `.loc[row_label, col_label]` accesses data by explicit index/column names. In Question 8, `df.iloc[0:2, :]` slices the first 2 rows (positions 0 and 1) across all columns.
- **Split-Apply-Combine Methodology (`groupby`):** Splits the table into distinct subgroups based on category keys (`Job`), applies an aggregation function to a numeric column (`["Salary"].mean()`), and combines the results into a compact summarized Series.

3. Complete Exam-Ready Algorithms (START ... STOP Format)

MAIN OVERALL ALGORITHM: Exercise 04 (All 12 Pandas Questions)

Step 1: START
Step 2: [Import Modules] Import `pandas as pd` and `numpy as np`.
Step 3: [Q1: List to Series] Define list `[1, 4, 6, 3]`. Convert via `pd.Series()` and print.
Step 4: [Q2: Date Range Generation] Generate daily timestamps from 2021-05-01 to 2021-05-12 using `pd.date_range()` and print.
Step 5: [Q3: Dict to DataFrame] Create dictionary of names, ages, cities. Convert via `pd.DataFrame()` and print.
Step 6: [Q4: 2D List to DataFrame] Create nested list of IDs, names, scores. Convert via `pd.DataFrame(listdata, columns=[...])` and print.
Step 7: [Q5: Read CSV] Ingest `medal.csv` via `pd.read_csv("medal.csv")` and print.
Step 8: [Q6: Multi-Column Sorting] Sort DataFrame by `Score` descending and `Name` ascending using `df.sort_values(by=["Score", "Name"], ascending=[False, True])`.
Step 9: [Q7: Reset Custom Index] Define DataFrame with index `["p1", "p2", "p3"]`. Reset index to default integers using `df.reset_index(drop=True)`.
Step 10: [Q8: Row Slicing via iloc] Select first two rows using integer location slicing: `df.iloc[0:2, :]`.
Step 11: [Q9: Groupby Salary Aggregation] Group DataFrame by occupation: `df.groupby("Job")["Salary"].mean()` and print average salaries.
Step 12: [Q10: Handle Missing Values] Ingest dictionary with `np.nan` using `from_dict(orient="index")`. Replace all NaNs with 0 using `df.fillna(0)`.
Step 13: [Q11: Boolean Mask Transformation] Convert profit column to boolean: `df["profit"] = df["profit"] > 0`.
Step 14: [Q12: Relational DataFrame Merging] Create two DataFrames with shared `ID` key. Merge via `pd.merge(df1, df2, how="inner", on="ID")` and print.
Step 15: STOP

4. Complete Theory, Formulations, and Worked Examples

4.1 Multi-Column Sorting Hierarchy

Given records: $[(1, \text{"Alice"}, 15), (2, \text{"Bob"}, 15), (3, \text{"Don"}, 45)]$:
Sorting rule: `by=["Score", "Name"], ascending=[False, True]`.
1. Primary Sort (Score descending): Score 45 comes first → Don (Score 45).
2. Secondary Sort (Name ascending on ties): Alice and Bob both have Score 15. Alphabetically, "Alice" < "Bob" → Alice precedes Bob.
Result order: Don (45), Alice (15), Bob (15).

4.2 Relational Merging Mathematics (Inner Join)

Let $R_1(\text{ID}, \text{Name}, \text{Stipend})$ and $R_2(\text{ID}, \text{Position})$:
$$\text{Inner Join } R_1 \bowtie_{\text{ID}} R_2 = \{ (t_1, t_2) \mid t_1 \in R_1, t_2 \in R_2 \text{ and } t_1.\text{ID} = t_2.\text{ID} \}$$ • Row 1: ID 101 matches → $(101, \text{"John"}, 30000, \text{"Developer"})$
• Row 2: ID 102 matches → $(102, \text{"Amith"}, 35000, \text{"Developer"})$
• Row 3: ID 103 matches → $(103, \text{"Sona"}, 20000, \text{"Tester"})$

4.3 Groupby Aggregation Mathematical Formulation

$$\text{Mean Salary for Job } j = \frac{1}{|S_j|} \sum_{i \in S_j} \text{Salary}_i$$ • Doctors: Bob (55,000) and Dylan (35,000) → $\text{Average} = \frac{55000 + 35000}{2} = 45,000.0$.
• Engineers: Emma (65,000) → $\text{Average} = 65,000.0$.
• Teachers: Alice (25,000) and Charlie (20,000) → $\text{Average} = \frac{25000 + 20000}{2} = 22,500.0$.

5. Complete Original Source Code and Block Walkthrough

5.1 Verbatim Source Code (`e04_pandas.py`)

```
# Exercise 4: Programs using Pandas

import pandas as pd
import numpy as np

# Question 1: Write a Python program to implement List-to-Series Conversion.

listdata = [1, 4, 6, 3]

series = pd.Series(listdata)

print(series)


# Question 2: Write a Python program to generate the series of dates from 1st May, 2021 to 12th May, 2021.

print(pd.date_range("2021-05-01", "2021-05-12"))


# Question 3: Given a dictionary, convert it into corresponding dataframe and display it.

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [24, 30, 28],
    "City": ["Delhi", "Mumbai", "Bangalore"]
}

df = pd.DataFrame(data)

print("Dictionary to DataFrame:")
print(df)


# Question 4: Given a 2D List, convert it into corresponding dataframe and display it.

listdata = [
    [1, "Alice", 10],
    [2, "Bob", 15],
    [3, "Don", 25]
]

df = pd.DataFrame(listdata, columns=["ID", "Name", "Score"])

print(df)


# Question 5: Given a CSV file, read it into a dataframe and display it.

df = pd.read_csv("medal.csv")

print(df)


# Question 6: Given a dataframe, sort it by multiple columns.

listdata = [
    [1, "Alice", 15],
    [2, "Bob", 15],
    [3, "Don", 45]
```

```
]

df = pd.DataFrame(listdata, columns=["ID", "Name", "Score"])

print("Data frame")
print(df)

sorted_df = df.sort_values(
    by=["Score", "Name"],
    ascending=[False, True]
)

print("Sorted Dataframe")
print(sorted_df)


# Question 7: Given a dataframe with custom indexing, convert it to default indexing and display it.

data = [
    ["Alice", 25, "New York"],
    ["Bob", 30, "Paris"],
    ["Charlie", 35, "London"]
]

df = pd.DataFrame(
    data,
    columns=["Name", "Age", "City"],
    index=["p1", "p2", "p3"]
)

print(df)

df = df.reset_index(drop=True)

print(df)


# Question 8: Given a dataframe, select first 2 rows and output them.

data = [
    ["Alice", 25, "New York"],
    ["Bob", 30, "Paris"],
    ["Charlie", 35, "London"]
]

df = pd.DataFrame(
    data,
    columns=["Name", "Age", "City"]
)

print(df.iloc[0:2, :])


# Question 9: Given is a dataframe showing name, occupation, salary of people. Find the average salary per
occupation.

data = [
    ["Alice", "Teacher", 25000],
    ["Bob", "Doctor", 55000],
    ["Charlie", "Teacher", 20000],
    ["Dylan", "Doctor", 35000],
    ["Emma", "Engineer", 65000]
]

df = pd.DataFrame(
```

```
    data,
    columns=["Name", "Job", "Salary"]
)

print(df)

avg_by_job = df.groupby("Job")["Salary"].mean()

print(avg_by_job)


# Question 10: Given a dataframe with NaN Values, fill the NaN values with 0.

data = {
    "A": [1, 2, np.nan, 4],
    "B": [np.nan, 2, 3, 4]
}

df = pd.DataFrame.from_dict(data, orient="index")

print(df)

df = df.fillna(0)

print(df)


# Question 11: Given is a dataframe showing Company Names and Profits. Convert positive profits to True and
the rest to False.

data = {
    "cname": ["A", "B", "C"],
    "profit": [1000, -500, 0]
}

df = pd.DataFrame(data)

df["profit"] = df["profit"] > 0

print("Profit Column Converted to Boolean:")
print(df)


# Question 12: Given are 2 dataframes containing Employee ID, Employee Name, Stipend and Position. Merge th
em using Employee ID.

data1 = [
    [101, "John", 30000],
    [102, "Amith", 35000],
    [103, "Sona", 20000]
]

df1 = pd.DataFrame(
    data1,
    columns=["ID", "Name", "Stipend"]
)

data2 = [
    [101, "Developer"],
    [102, "Developer"],
    [103, "Tester"]
]

df2 = pd.DataFrame(
    data2,
```

```
    columns=["ID", "Position"]
)

df3 = pd.merge(
    df1,
    df2,
    how="inner",
    on="ID"
)

print(df3)
```

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. `pd.Series(data, index=None)` **:** Constructs a 1D labeled array. Automatically generates default integer indices $(0, 1, 2, \dots)$. Line 10 converts Python list to Series.
2. `pd.date_range(start, end, freq='D')` **:** Returns a DatetimeIndex with fixed frequency (default daily 'D'). Both start and end dates are inclusive.
3. `pd.DataFrame(data, columns=None, index=None)` **:** Primary 2D tabular structure. Ingests dictionaries (keys become column names) or 2D nested lists (where `columns=[...]` defines headers).
4. `pd.read_csv(filepath)` **:** High-performance parser reading CSV files into DataFrames, auto-detecting headers and data types. Reads `medal.csv` in Line 49.
5. `df.sort_values(by, ascending=True)` **:** Sorts DataFrame rows by specified column names. Line 67 passes lists to both arguments: `by=["Score", "Name"]` and `ascending=[False, True]` for mixed directional sorting.
6. `df.reset_index(drop=True)` **:** Resets row labels back to default integer range $(0, 1, \dots)$. Parameter `drop=True` prevents the old custom index from being inserted as a new column in the DataFrame.
7. `df.iloc[row_indexer, col_indexer]` **:** Purely integer-location based indexing for selection by position. `df.iloc[0:2, :]` extracts rows at positions 0 and 1 across all columns.
8. `df.groupby(by)[column].mean()` **:** Performs Split-Apply-Combine aggregation. Groups rows by identical values in `Job`, selects `Salary`, and calculates the arithmetic mean for each job category.
9. `pd.DataFrame.from_dict(data, orient='index')` **:** Alternative DataFrame constructor. `orient='index'` treats dictionary keys as row index labels rather than column names.
10. `df.fillna(value)` **:** Imputes missing NaN (Not-a-Number) values with a specified replacement scalar (Line 146 replaces NaNs with 0).
11. **Boolean Column Masking** `df["col"] > 0` **:** Applies element-wise comparison across the Series, producing a boolean Series (True for positive profits, False for zero or negative). Reassigns in-place in Line 160.
12. `pd.merge(left, right, how='inner', on=None)` **:** Database-style relational merge. `how='inner'` retains only rows where the join key (`on="ID"`) exists in both DataFrames.

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
`pd.Series()` Constructor	Line 10	✓ Verified	1D array conversion
`pd.date_range()` Generation	Line 17	✓ Verified	Temporal timestamp indexing
Dictionary to DataFrame	Line 28	✓ Verified	Key-to-column tabular creation
2D Nested List with Column Labels	Line 42	✓ Verified	Row-based matrix creation
`pd.read_csv("medal.csv")`	Line 49	✓ Verified	External file ingestion
Multi-Column Composite Sorting	Lines 67–70	✓ Verified	Mixed `ascending=[False, True]`
`reset_index(drop=True)`	Line 92	✓ Verified	Index flattening without column retention
Positional Slicing via `.iloc[0:2, :]`	Line 110	✓ Verified	Integer location row selection
`groupby().mean()` Aggregation	Line 130	✓ Verified	Grouped summary statistics
`from_dict(orient="index")` & `fillna(0)`	Lines 142, 146	✓ Verified	Orientation parsing & NaN imputation
In-place Boolean Column Masking	Line 160	✓ Verified	Column-level threshold conversion
Relational Database Merge `pd.merge()`	Lines 190–195	✓ Verified	Inner join on shared key

7. Step-by-Step Program Execution Walkthrough

Exercise	Operation & Expression	Data Transformed	Output Structure
Q1	`pd.Series([1, 4, 6, 3])`	List of 4 integers	1D Series with index 0 to 3, dtype int64
Q2	`pd.date_range("2021-05-01", "2021-05-12")`	12 consecutive dates	DatetimeIndex of length 12, freq='D'
Q3	`pd.DataFrame(dict)`	3 names, ages, cities	3×3 DataFrame with named columns
Q4	`pd.DataFrame(2D_list, columns=...)`	3 rows of [ID, Name, Score]	3×3 DataFrame
Q5	`pd.read_csv("medal.csv")`	5 countries and gold medals	5×2 DataFrame
Q6	`sort_values(by=[...], ascending=[...])`	Scores 15, 15, 45	Sorted: Don (45), Alice (15), Bob (15)
Q7	`reset_index(drop=True)`	Index p1, p2, p3 dropped	Default integer index 0, 1, 2 restored
Q8	`df.iloc[0:2, :]`	Rows 0 and 1 extracted	2 rows (Alice and Bob) across 3 columns
Q9	`groupby("Job")["Salary"].mean()`	5 employee records grouped by 3 jobs	Series: Doctor 45k, Engineer 65k, Teacher 22.5k
Q10	`from_dict(orient="index").fillna(0)`	Dictionary with NaNs transposed	Missing values replaced with 0.0
Q11	`df["profit"] > 0`	Profits [1000, -500, 0]	Boolean column: [True, False, False]
Q12	`pd.merge(df1, df2, on="ID")`	3 records merged by Employee ID	Combined DataFrame with 4 columns

8. Output and Practical Interpretation

8.1 Verified Terminal Output

```
0    1
1    4
2    6
3    3
dtype: int64
DatetimeIndex(['2021-05-01', '2021-05-02', '2021-05-03', '2021-05-04',
               '2021-05-05', '2021-05-06', '2021-05-07', '2021-05-08',
               '2021-05-09', '2021-05-10', '2021-05-11', '2021-05-12'],
              dtype='datetime64[ns]', freq='D')
Dictionary to DataFrame:
      Name  Age       City
0    Alice   24      Delhi
1      Bob   30     Mumbai
2  Charlie   28  Bangalore
   ID   Name  Score
0   1  Alice     10
1   2    Bob     15
2   3    Don     25
   Country  Gold_Medal
0      USA          46
1       UK          27
2    China          26
3   Russia          19
4  Germany          17
Data frame
   ID   Name  Score
0   1  Alice     15
1   2    Bob     15
2   3    Don     45
Sorted Dataframe
   ID   Name  Score
2   3    Don     45
0   1  Alice     15
1   2    Bob     15
      Name  Age      City
p1   Alice   25  New York
p2     Bob   30     Paris
p3  Charlie   35    London
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London
    Name  Age      City
0  Alice   25  New York
1    Bob   30     Paris
      Name       Job  Salary
0    Alice   Teacher   25000
1      Bob    Doctor   55000
2  Charlie   Teacher   20000
3    Dylan    Doctor   35000
4     Emma  Engineer   65000
Job
Doctor      45000.0
Engineer    65000.0
Teacher     22500.0
Name: Salary, dtype: float64
     0    1    2    3
A  1.0  2.0  NaN  4.0
B  NaN  2.0  3.0  4.0
```

```
     0    1    2    3
A  1.0  2.0  0.0  4.0
B  0.0  2.0  3.0  4.0
Profit Column Converted to Boolean:
  cname  profit
0     A    True
1     B   False
2     C   False
    ID   Name  Stipend   Position
0  101   John    30000  Developer
1  102  Amith    35000  Developer
2  103   Sona    20000     Tester
```

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

EXERCISE 4: PROGRAMS USING PANDAS

AIM: To write a Python program using Pandas to perform data wrangling tasks including Series creation, date ranges, DataFrame generation from dicts and 2D lists, CSV ingestion, multi-column sorting, index resetting, iloc row slicing, groupby aggregation, NaN imputation, boolean masking, and relational merging.

ALGORITHM:
1. Import `pandas as pd` and `numpy as np`.
2. Create Series using `pd.Series()` and date ranges via `pd.date_range()`.
3. Convert dictionaries and 2D lists to DataFrames with explicit headers.
4. Ingest CSV file data using `pd.read_csv("medal.csv")`.
5. Sort records on multiple columns using `df.sort_values(by=[...], ascending=[...])`.
6. Convert custom index back to default integers using `df.reset_index(drop=True)`.
7. Select specific rows using integer slicing `df.iloc[0:2, :]`.
8. Aggregate average salary per occupation using `df.groupby("Job")["Salary"].mean()`.
9. Impute missing values with 0 using `df.fillna(0)` on oriented dictionaries.
10. Apply boolean mask `df["profit"] > 0` and merge tables on employee ID via `pd.merge()`.

RESULT: All 12 Pandas data manipulation exercises were successfully executed and verified.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** What does `drop=True` do in `df.reset_index(drop=True)`?
  Answer: It discards the previous custom index labels completely. Without `drop=True`, the old index is retained and inserted as a new column named `index` in the DataFrame.
- **3-Mark Question:** Differentiate between `.loc` and `.iloc` in Pandas.
  Answer: `.loc` is label-based indexing where rows and columns are selected by their literal index names or column string headers. In contrast, `.iloc` is purely integer-position based indexing where rows and columns are accessed strictly by 0-based integer coordinates ($0, 1, \dots$).
- **5-Mark Question:** Explain the Split-Apply-Combine strategy used in `df.groupby("Job")["Salary"].mean()`.
  Answer: The expression performs three distinct phases: (1) **Split:** Divides DataFrame rows into separate groups based on distinct values in the `Job` column (Doctor, Engineer, Teacher). (2) **Apply:** Extracts the `Salary` column for each independent group and computes the arithmetic mean. (3) **Combine:** Merges the computed means into a single returned Pandas Series indexed by the unique job categories.

9.3 Predict-the-Output Questions

1. **Question:** What is the output of `pd.Series([10, -5, 0]) > 0`?
   Answer: A boolean Series containing `[True, False, False]`.
2. **Question:** How many elements are generated by `pd.date_range("2023-01-01", "2023-01-05")`?
   Answer: 5 daily timestamps (both start and end dates are inclusive in Pandas date ranges).

9.4 Identify-the-Function Questions

1. **Question:** Which Pandas function replaces missing NaN values with specified constants?
   Answer: `df.fillna()`.
2. **Question:** Which function performs relational database joins between two DataFrames on a common key?
   Answer: `pd.merge()`.
3. **Question:** Which function sorts a DataFrame by one or more specified columns?
   Answer: `df.sort_values()`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: What is the fundamental difference between a Pandas Series and a DataFrame?
Answer: A Series is a 1-dimensional labeled homogeneous array, whereas a DataFrame is a 2-dimensional labeled tabular data structure where each column is internally a Series.

Q2: How does Pandas handle missing values internally?
Answer: Missing values are represented using `np.nan` (IEEE 754 floating-point Not-a-Number), which allows numerical arrays to support missing data without altering data alignment.

Q3: What happens when `orient="index"` is passed to `pd.DataFrame.from_dict()`?
Answer: Dictionary keys become row index labels, and dictionary value lists become horizontal rows rather than vertical columns.

Q4: Why does `df.iloc[0:2, :]` return 2 rows instead of 3?
Answer: Like standard Python slice notation, `.iloc[start:stop]` uses a half-open interval $[start, stop)$ where the stop index is exclusive. Thus `0:2` selects rows at positions 0 and 1.

Q5: What are the supported join types in `pd.merge()` via the `how` argument?
Answer: `'inner'` (default, only matching keys), `'left'` (all left keys), `'right'` (all right keys), and `'outer'` (union of all keys from both tables).

Q6: How do you sort a DataFrame by multiple columns in opposite directions?
Answer: By passing lists to both `by` and `ascending`: `df.sort_values(by=["ColA", "ColB"], ascending=[False, True])`.

Q7: Does `df.fillna(0)` modify the original DataFrame in place by default?
Answer: No. By default, it returns a new modified copy. To modify in place, pass `inplace=True` or reassign: `df = df.fillna(0)`.

Q8: What is the frequency of timestamps generated by `pd.date_range()` by default?
Answer: Calendar daily frequency (`freq='D'`).

Q9: How do you access a single column in a DataFrame?
Answer: Using bracket notation `df["ColumnName"]` or dot attribute notation `df.ColumnName` (if the name is a valid Python identifier without spaces).

Q10: What does `df.shape` return for a DataFrame with 5 rows and 3 columns?
Answer: The tuple `(5, 3)`.

Q11: Why is Pandas essential for machine learning pipelines?
Answer: Machine learning algorithms require structured numerical inputs. Pandas handles initial data loading, missing value imputation, categorical encoding, filtering, merging, and column slicing before converting to NumPy matrices for model fitting.

Q12: What happens if two DataFrames in `pd.merge()` have conflicting column names not used as keys?
Answer: Pandas automatically appends suffixes: `_x` for the left table column and `_y` for the right table column (configurable via the `suffixes` parameter).

Q13: How does boolean masking work when filtering rows in Pandas?
Answer: Evaluating a condition like `df["Age"] > 25` produces a boolean Series. Passing this into `df[df["Age"] > 25]` filters and extracts only rows where the condition evaluates to True.

Q14: What is the return type of `df.groupby("Job")["Salary"].mean()`?
Answer: A Pandas Series where the index contains the unique job names and the values are the computed floating-point mean salaries.

Q15: Can `pd.read_csv()` parse custom delimiters like semicolons or tabs?
Answer: Yes, by specifying the `sep` or `delimiter` parameter, e.g., `pd.read_csv("file.txt", sep="\t")`.

Q16: How do you convert a Pandas Series or DataFrame column to a NumPy ndarray?
Answer: Using the `.to_numpy()` method or the `.values` attribute: `df["Col"].to_numpy()`.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	Correction / Debugging Technique
Old index appearing as new column after reset	Calling `df.reset_index()` without `drop=True`.	Always use `df.reset_index(drop=True)` if you do not want to keep the old index.
`KeyError` when accessing columns in `.loc`	Misspelling column names or confusing `.loc` with `.iloc`.	Use exact column names with `.loc`; use 0-based integer positions with `.iloc`.
`fillna()` result not reflected in DataFrame	Forgetting that Pandas methods return copies by default.	Reassign the result: `df = df.fillna(0)` or specify `inplace=True`.
Merge fails with `KeyError: 'ID'`	Join key specified in `on="ID"` does not exist in both DataFrames.	Verify that both DataFrames have identical key spelling, or use `left_on` and `right_on`.
Date range includes unexpected extra day	Forgetting that Pandas `date_range` end dates are inclusive.	Specify the exact final date desired (both endpoints are included).

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Mnemonics & Memory Aids

- **Mnemonic for Indexing — "L for Label, I for Integer":**
  `.loc` → **L**abel-based selection • `.iloc` → **I**nteger-location selection.
- **Mnemonic for Groupby: S-A-C** — **S**plit → **A**pply → **C**ombine.
- **Reset Index Rule:** "Always Drop the Old" — `df.reset_index(drop=True)`.

12.2 Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. What is the difference between `.loc` and `.iloc`?
Answer: `.loc` selects by label names; `.iloc` selects strictly by 0-based integer positions.

2. Why use `drop=True` in `reset_index()`?
Answer: To discard the former index and prevent it from becoming an unwanted extra column.

3. How does `df.groupby("Job")["Salary"].mean()` work?
Answer: Splits data by Job, extracts Salary, calculates group averages, and combines into a Series.

4. What does `how='inner'` mean in `pd.merge()`?
Answer: Retains only matching rows where the key exists in both the left and right DataFrames.

5. How are missing values represented in Pandas?
Answer: As `np.nan` (Not-a-Number); replaced using `df.fillna(value)`.

13. Cross-Program Understanding (Data Science Pipeline Progression)

Pandas serves as the central data wrangling bridge between raw storage and machine learning algorithms:

Pipeline Stage	Pandas Operation in Exercise 04	Subsequent Role in Exercises 05 to 10
1. Ingestion	`pd.read_csv("medal.csv")`	Loading tabular datasets from disk (used directly in Exercise 03 Q9)
2. Cleaning	`df.fillna(0)`	Handling missing attributes in real-world clinical and survey datasets
3. Feature Slicing	`df.iloc[0:2, :]`	Separating feature matrices X from target vectors y in ML models
4. Transformation	`df["profit"] > 0`	Binarizing continuous targets for binary classification
5. Aggregation	`df.groupby().mean()`	Evaluating class balances and exploratory cluster profiling